

R Programming Practice (Part 5): Functions

Tim Irwin

2026-04-27

R Programming Practice (Part 5): Functions

This practice set covers functions from *R Programming for Data Science* by Roger D. Peng (pages 56–64):

1. Basic function syntax
2. Default arguments and argument matching
3. Functions that operate on vectors
4. Functions with control flow (combining if/else and loops from the last practice set)
5. Functions that operate on data frames
6. Advanced topics: lazy evaluation, NULL defaults, the `...` argument

Each section starts with a **lesson** that teaches the concept with a worked example, followed by **exercises** that apply what you just learned. The exercises **build on each other** — each one extends or combines the previous ones.

You may want to start with:

```
rm(list = ls())
```

Tip: Build functions incrementally. Get the simplest version working first, then add features one at a time. If a function isn't working, test the body of it line by line in the console with concrete values before wrapping it back up.

Datasets used in this practice

- **airquality** — Daily air quality measurements in New York, May–September 1973.
 - **mtcars** — Fuel consumption and 10 aspects of design/performance for 32 cars.
 - **iris** — Measurements of 150 iris flowers across 3 species.
-

1. Basic Functions

Lesson 1.1: Function syntax

In R, functions are created with the `function()` directive and stored as objects, just like vectors or data frames. The general structure is:

```
my_function <- function(arg1, arg2) {  
  # body - code that does something  
}
```

Here's a complete, working example. This function takes no arguments and just prints a message to the console:

```
welcome <- function() {
  print("Welcome to R!")
}

welcome()
```

```
## [1] "Welcome to R!"
```

Things to notice:

- We define the function with `function() { ... }` and assign it to `welcome`.
- We **call** the function with `welcome()` — the empty parentheses are required even when there are no arguments.
- The curly braces `{}` enclose the body. R will run everything inside them when the function is called.
- Functions are themselves R objects. You can confirm this with `class(welcome)` — it returns `"function"`.

Exercise 1.1

Write a function called `say_hello` that takes no arguments and prints `"Hello, world!"`. Call it.

```
say_hello <- function() {
  print("Hello, world!")
}

say_hello()
```

```
## [1] "Hello, world!"
```

Lesson 1.2: Adding arguments

Functions become useful when they take **arguments** — inputs the function works with. Each argument acts like a placeholder variable available inside the function body.

Here's `welcome` extended to accept a name:

```
welcome_user <- function(user) {
  message <- paste0("Welcome, ", user, "!")
  print(message)
}

welcome_user("Roger")
```

```
## [1] "Welcome, Roger!"
```

```
welcome_user("Maya")
```

```
## [1] "Welcome, Maya!"
```

`paste0()` glues strings together with no separator between them. `paste()` does the same but adds a space by default — both are useful, depending on what you want.

Exercise 1.2

Modify your `say_hello` function from Exercise 1.1 to take a `name` argument and print `"Hello, <name>!"` (substituting the actual name). Test it with your own name.

```
say_hello <- function(name) {
  message <- paste0("Hello ", name, "!")
  print(message)
}
say_hello("Tim")
```

```
## [1] "Hello Tim!"
```

Lesson 1.3: Return vs print

This is one of the most important concepts about functions. **Printing** shows something on the screen but doesn't make it available for use later. **Returning** gives a value back to whatever called the function so it can be assigned, passed along, or used in another calculation.

In R, the return value of a function is always the value of the **last expression** in the body. You typically don't need to write `return()` explicitly.

Compare these two functions:

```
prints_only <- function(n) {
  print(n * 3)
}
```

```
returns_value <- function(n) {
  n * 3
}
```

```
a <- prints_only(7)  # 21 prints, but `a` doesn't get the value
```

```
## [1] 21
```

```
a                                     # NULL - nothing useful was returned
```

```
## [1] 21
```

```
b <- returns_value(7) # nothing prints, but `b` gets 21
b
```

```
## [1] 21
```

Tip: When in doubt, **return** values rather than print them. Returned values can be assigned, passed to other functions, or used in arithmetic. Printed values can only be looked at.

Exercise 1.3

Modify your function from Exercise 1.2 again so that instead of printing the greeting, it **returns** it as a string. Store the result in a variable called `msg` and then print `msg`.

```
say_hello <- function(name) {
  message <- paste0("Hello ", name, "!")
}
msg <- say_hello("Tim")
msg
```

```
## [1] "Hello Tim!"
```

Lesson 1.4: Multiple arguments

Functions can take any number of arguments, separated by commas. Here's one that takes two:

```
area_of_rectangle <- function(length, width) {  
  length * width  
}
```

```
area_of_rectangle(4, 5)
```

```
## [1] 20
```

```
area_of_rectangle(10, 3)
```

```
## [1] 30
```

The body uses both arguments, and the result of `length * width` is the last expression — so it gets returned automatically.

Exercise 1.4

Write a function called `double_it` that takes a number `x` and returns `x` multiplied by 2. Test it with `double_it(5)` and `double_it(-3)`.

```
double_it <- function(x) {  
  x * 2  
}
```

```
double_it(5)
```

```
## [1] 10
```

```
double_it(-3)
```

```
## [1] -6
```

Exercise 1.5

Generalize Exercise 1.4: write `multiply_by(x, factor)` that takes two numbers and returns their product. Test it with `multiply_by(5, 3)`.

```
multiply_by <- function(x, factor) {  
  x * factor  
}
```

```
multiply_by(5, 3)
```

```
## [1] 15
```

2. Default Arguments and Argument Matching

Lesson 2.1: Default values

You can give arguments **default values** in the function definition. If the caller doesn't supply a value for that argument, R uses the default. Defaults are set with `=` inside the `function()` parentheses:

```
celsius_to_fahrenheit <- function(c, precision = 1) {  
  result <- (c * 9/5) + 32  
  round(result, precision)
```

```

}

celsius_to_fahrenheit(20)           # uses default precision of 1 → 68

## [1] 68
celsius_to_fahrenheit(20, precision = 3)  # overrides default → 68.000

## [1] 68
celsius_to_fahrenheit(36.5)           # body temperature → 97.7

## [1] 97.7

```

Defaults are useful when an argument is *usually* the same value, but you occasionally want to override it. This relieves the user from having to specify every argument every time.

Exercise 2.1

Take your `multiply_by` function from Exercise 1.5 and add a default value of 2 for `factor`. Now `multiply_by(5)` should return 10, but `multiply_by(5, 3)` should still return 15.

```

multiply_by <- function(x, factor = 2) {
  x * factor
}

multiply_by(5)

## [1] 10
multiply_by(5, 3)

## [1] 15

```

Lesson 2.2: How R matches arguments

R is flexible about how you supply arguments. Given a function `f(x, y)`, you can call it:

- **Positionally:** `f(1, 2)` — first value goes to `x`, second to `y`
- **By name:** `f(x = 1, y = 2)` — explicit assignment
- **By name in any order:** `f(y = 2, x = 1)` — order doesn't matter when using names
- **Mixed:** `f(1, y = 2)` — positional first, then named

```

power <- function(base, exponent) {
  base ^ exponent
}

power(2, 10)           # positional

## [1] 1024
power(base = 2, exponent = 10)  # named

## [1] 1024
power(exponent = 10, base = 2)  # named, reverse order

## [1] 1024

```

```
power(2, exponent = 10) # mixed
```

```
## [1] 1024
```

All four return the same answer (1024).

Tip: Named arguments make code more readable, especially when a function has many arguments or when the name clarifies what a value represents. For functions with just one or two arguments, positional is fine.

Exercise 2.2

Call your `multiply_by` function from Exercise 2.1 four different ways and verify they all give the same answer:

1. Positional: `multiply_by(5, 3)`
2. Named: `multiply_by(x = 5, factor = 3)`
3. Named, reverse order: `multiply_by(factor = 3, x = 5)`
4. Mixed: `multiply_by(5, factor = 3)`

```
multiply_by(5, 3)
```

```
## [1] 15
```

```
multiply_by(x = 5, factor = 3)
```

```
## [1] 15
```

```
multiply_by(factor = 3, x = 5)
```

```
## [1] 15
```

```
multiply_by(5, factor = 3)
```

```
## [1] 15
```

Lesson 2.3: Required vs optional arguments

Arguments without a default are **required** — the function will throw an error if they're missing. Arguments with defaults are **optional**. R's convention is to put required arguments first and optional ones (with defaults) after.

```
order_total <- function(price, quantity, tax_rate = 0.08) {  
  subtotal <- price * quantity  
  subtotal + (subtotal * tax_rate)  
}
```

```
order_total(20, 3) # uses default tax_rate
```

```
## [1] 64.8
```

```
order_total(20, 3, tax_rate = 0.10) # overrides default
```

```
## [1] 66
```

If you call `order_total(20)` without supplying `quantity`, R will error with "argument 'quantity' is missing, with no default".

Exercise 2.3

Write a function called `discount_price(price, discount_pct = 10)` that returns the price after applying a percentage discount. So `discount_price(100)` should return 90, and `discount_price(100, 25)` should return 75.

```
discount_price <- function(price, discount_pct = 10) {  
  price - discount_pct  
}  
discount_price(100)
```

```
## [1] 90
```

```
discount_price(100, 25)
```

```
## [1] 75
```

3. Functions That Operate on Vectors

Lesson 3.1: Functions and vectors

Most useful R functions operate on whole vectors, not single values. When you write `mean(x)`, `x` is a whole vector. Your custom functions should usually be designed the same way.

Here's a function that uses a `for` loop to compute the **product** of all elements in a vector:

```
my_product <- function(x) {  
  result <- 1 # start at 1, not 0 (multiplying by 0 zeros everything)  
  for (i in seq_along(x)) {  
    result <- result * x[i]  
  }  
  result  
}
```

```
my_product(c(2, 3, 4)) # 2 * 3 * 4 = 24
```

```
## [1] 24
```

```
my_product(c(5, 10)) # 50
```

```
## [1] 50
```

The pattern: initialize an accumulator (`result <- 1`), loop over the vector, update the accumulator each iteration, and the final value is the result. This is the same loop pattern you used in the control structures practice — just wrapped in a function so you can reuse it.

Exercise 3.1

Write a function called `vec_sum(x)` that takes a numeric vector and returns the sum of its elements. **Don't use `sum()`** — write it with a `for` loop. Test it with `c(1, 2, 3, 4, 5)` (should return 15).

```
vec_sum <- function(x) {  
  result <- 0  
  for (i in seq_along(x)) {  
    result <- result + x[i]  
  }  
  result
```

```

}

vec_sum(c(1, 2, 3, 4, 5))

## [1] 15

```

Lesson 3.2: Adding NA handling with arguments

Real data has missing values. A common pattern is to add an `na.rm` argument that controls whether NAs are skipped. R's built-in functions like `mean()` and `sum()` follow this convention.

```

my_max <- function(x, na.rm = FALSE) {
  best <- -Inf # start at negative infinity
  for (i in seq_along(x)) {
    if (is.na(x[i])) {
      if (na.rm) {
        next # skip this value
      } else {
        return(NA) # if any NA and na.rm=FALSE, result is NA
      }
    }
    if (x[i] > best) {
      best <- x[i]
    }
  }
  best
}

my_max(c(3, 7, NA, 4)) # NA (default behavior)

## [1] NA

my_max(c(3, 7, NA, 4), na.rm = TRUE) # 7

```

```
## [1] 7
```

The `if (is.na(x[i]))` check catches NAs *before* trying to use them in arithmetic, which would otherwise produce errors or propagate NAs.

Exercise 3.2

Modify `vec_sum` from Exercise 3.1 to handle NAs. Add an argument `na.rm` that defaults to `FALSE`. If `TRUE`, skip NA values. If `FALSE` and there are NAs, return `NA`. Test it with `c(1, 2, NA, 4, 5)` both ways.

```

vec_sum <- function(x, na.rm = FALSE) {
  result <- 0
  for (i in seq_along(x)) {
    if (is.na(x[i])) {
      if (na.rm) {
        next
      } else {
        return(NA)
      }
    }
    result <- result + x[i]
  }
}

```



```

    }
    result
  }

vec_sum(c(1, 2, NA, 4, 5))

## [1] NA
vec_sum(c(1, 2, NA, 4, 5), na.rm = TRUE)

## [1] 12

```

Exercise 3.3

Test your `vec_sum` from Exercise 3.2 on `airquality$Ozone` with `na.rm = TRUE`. Compare your result to `sum(airquality$Ozone, na.rm = TRUE)` — they should match.

```

vec_sum(airquality$Ozone, na.rm = TRUE)

## [1] 4887
sum(airquality$Ozone, na.rm = TRUE)

## [1] 4887

```

Lesson 3.3: Returning multiple values with a named list

What if your function needs to return more than one thing? You can't return multiple separate values, but you *can* wrap them in a named list:

```

spread_info <- function(x) {
  list(
    smallest = min(x, na.rm = TRUE),
    largest = max(x, na.rm = TRUE),
    range_size = max(x, na.rm = TRUE) - min(x, na.rm = TRUE)
  )
}

result <- spread_info(c(15, 22, 8, 31, 14))
result      # see the whole list

## $smallest
## [1] 8
##
## $largest
## [1] 31
##
## $range_size
## [1] 23

result$smallest      # extract a specific value

## [1] 8
result$range_size

## [1] 23

```

The names act as labels, so the caller knows what each value represents and can extract specific ones with `$`. This is the standard R idiom for returning multiple values from one function.

Exercise 3.4

Write a function called `vec_summary(x)` that takes a numeric vector and returns a named list with five elements: `mean`, `median`, `min`, `max`, and `n_missing` (the count of NAs). Make sure the mean/median/min/max calculations exclude NAs.

```
vec_summary <- function(x) {  
  list(  
    mean = mean(x, na.rm = TRUE),  
    median = median(x, na.rm = TRUE),  
    min = min(x, na.rm = TRUE),  
    max = max(x, na.rm = TRUE),  
    n_missing = sum(is.na(x))  
  )  
}
```

Exercise 3.5

Apply `vec_summary` from Exercise 3.4 to two different columns: `airquality$Wind` and `airquality$Ozone`. Then extract just the `mean` from each result using `$`.

```
vec_summary(airquality$Wind)
```

```
## $mean  
## [1] 9.957516  
##  
## $median  
## [1] 9.7  
##  
## $min  
## [1] 1.7  
##  
## $max  
## [1] 20.7  
##  
## $n_missing  
## [1] 0
```

```
vec_summary(airquality$Ozone)
```

```
## $mean  
## [1] 42.12931  
##  
## $median  
## [1] 31.5  
##  
## $min  
## [1] 1  
##  
## $max  
## [1] 168  
##  
## $n_missing
```

```
## [1] 37
vec_summary(airquality$Wind)$mean

## [1] 9.957516
vec_summary(airquality$Ozone)$mean

## [1] 42.12931
```

4. Functions with Control Flow

This section combines functions with the if/else and loops you practiced in Part 4.

Lesson 4.1: if/else inside functions

Putting if/else inside a function lets you make decisions based on the input. The pattern is the same as standalone if/else — it's just inside the function body:

```
weather_advice <- function(temp) {
  if (temp > 85) {
    "Stay hydrated and seek shade"
  } else if (temp < 40) {
    "Bundle up - it's cold"
  } else {
    "Comfortable weather"
  }
}

weather_advice(92)
```

```
## [1] "Stay hydrated and seek shade"
```

```
weather_advice(35)
```

```
## [1] "Bundle up - it's cold"
```

```
weather_advice(70)
```

```
## [1] "Comfortable weather"
```

Notice there's no explicit `return()` — whichever branch executes, its value is the last expression evaluated, so that's what gets returned.

Exercise 4.1

Write a function called `classify(x)` that takes a single number and returns:

- "positive" if $x > 0$
- "negative" if $x < 0$
- "zero" if $x == 0$

Test it with several values.

```
classify <- function(x) {
  if (x > 0) {
    "positive"
  } else if (x < 0) {
```

```

    "negative"
  } else {
    "zero"
  }
}
classify(9)

## [1] "positive"
classify(-12)

## [1] "negative"
classify(0)

## [1] "zero"

```

Lesson 4.2: Handling NA inside functions

You hit this in your control structures practice: comparisons with NA produce NA, which can break if statements. The fix is to **check for NA first** before doing any other comparison:

```

rate_speed <- function(mph) {
  if (is.na(mph)) {
    return("unknown")      # early return - done
  }

  if (mph > 70) {
    "fast"
  } else if (mph < 30) {
    "slow"
  } else {
    "moderate"
  }
}

rate_speed(80)

## [1] "fast"
rate_speed(25)

## [1] "slow"
rate_speed(NA)

## [1] "unknown"

```

This uses an explicit `return()` for the NA case so the function exits immediately. After that, the rest of the function can safely assume `mph` is a real number.

Exercise 4.2

Modify `classify` from Exercise 4.1 to also handle NA. If `x` is missing, return `"missing"`. Check `is.na()` *first*, before the other comparisons.

```

classify <- function(x) {
  if (is.na(x)){
    return("missing")
  }

  if (x > 0) {
    "positive"
  } else if (x < 0) {
    "negative"
  } else {
    "zero"
  }
}

```

```
classify(9)
```

```
## [1] "positive"
```

```
classify(-12)
```

```
## [1] "negative"
```

```
classify(0)
```

```
## [1] "zero"
```

```
classify(NA)
```

```
## [1] "missing"
```

Lesson 4.3: Vectorizing — making it work on whole vectors

The functions above take a single number. To apply that logic to a whole vector, you can write a second function that loops over the vector and calls the single-value function on each element:

```

rate_speed <- function(mph) {
  if (is.na(mph)) return("unknown")
  if (mph > 70) "fast"
  else if (mph < 30) "slow"
  else "moderate"
}

# Vectorized version
rate_speed_all <- function(mphs) {
  results <- character(length(mphs))      # pre-allocate result vector
  for (i in seq_along(mphs)) {
    results[i] <- rate_speed(mphs[i])     # call inner function
  }
  results
}

rate_speed_all(c(80, 25, 55, NA, 10))

```

```
## [1] "fast"      "slow"      "moderate" "unknown"  "slow"
```

Two patterns to notice:

1. **Pre-allocate** the result vector with `character(length(mphs))`. This is more efficient than growing a vector inside a loop.
2. **Calling your own function from inside another function** is encouraged — it keeps each function small and focused on one job.

Exercise 4.3

Write `classify_all(x)` that takes a **vector** of numbers and returns a **vector** of classifications. Use a `for` loop internally, and call your `classify` function from Exercise 4.2 inside the loop.

Test it with `c(5, -3, 0, NA, 10)` — you should get back `c("positive", "negative", "zero", "missing", "positive")`.

```
# Your code here
classify <- function(x) {
  if (is.na(x)) return("missing")
  if (x > 0) "positive"
  else if (x < 0) "negative"
  else "zero"
}

classify_all <- function(x) {
  results <- character(length(x))
  for (i in seq_along(x)) {
    results[i] <- classify(x[i])
  }
  results
}

(classify_all(c(5, -3, 0, NA, 10)))

## [1] "positive" "negative" "zero"      "missing"  "positive"
```

Exercise 4.4

Apply `classify_all` from Exercise 4.3 to `airquality$Temp - 75` (each day's temperature relative to 75°F). Use `table()` on the result to see how many days were above, at, or below 75.

```
table(classify_all(airquality$Temp - 75))

##
## negative positive      zero
##      48      101         4
```

Exercise 4.5

Write a function called `categorize_mpg(x)` that takes a numeric vector of mpg values and returns a character vector classifying each:

- "efficient" if `mpg > 25`
- "average" if `15 ≤ mpg ≤ 25`
- "thirsty" if `mpg < 15`

Apply it to `mtcars$mpg` and use `table()` to see the counts.

```
mpg <- function(x) {
  if (is.na(x)) return("missing")
  if (x > 25) "efficient"
```

```

    else if (x < 15) "thirsty"
    else "average"
  }

categorize_mpg <- function(x) {
  results <- character(length(x))
  for (i in seq_along(x)) {
    results[i] <- mpg(x[i])
  }
  results
}

categorize_mpg(mtcars$mpg)

## [1] "average" "average" "average" "average" "average" "average"
## [7] "thirsty" "average" "average" "average" "average" "average"
## [13] "average" "average" "thirsty" "thirsty" "thirsty" "efficient"
## [19] "efficient" "efficient" "average" "average" "average" "thirsty"
## [25] "average" "efficient" "efficient" "efficient" "average" "average"
## [31] "average" "average"

table(categorize_mpg(mtcars$mpg))

##
## average efficient thirsty
##      21         6         5

```

5. Functions That Operate on Data Frames

Lesson 5.1: Accessing a column by a variable

When you have the column name in a variable (rather than typing it literally), you need to use `[[ ]]`, not `$`:

```

get_column <- function(df, col) {
  df[[col]]      # works - uses the value of `col`
  # df$col       # WRONG - looks for a literal column called "col"
}

head(get_column(mtcars, "mpg"))

## [1] 21.0 21.0 22.8 21.4 18.7 18.1

head(get_column(iris, "Species"))

```

```

## [1] setosa setosa setosa setosa setosa setosa
## Levels: setosa versicolor virginica

```

The `$` operator only works with literal names — it doesn't evaluate variables. This is one of the most common gotchas when writing functions that work with data frames.

Exercise 5.1

Write a function called `column_mean(df, col)` where `df` is a data frame and `col` is a column name as a character string. The function should return the mean of that column with NAs removed. Test it on `mtcars`

with "mpg", and on airquality with "Ozone".

```
column_mean <- function(df, col) {  
  mean(df[[col]], na.rm = TRUE)  
}  
column_mean(mtcars, "mpg")
```

```
## [1] 20.09062
```

```
column_mean(airquality, "Ozone")
```

```
## [1] 42.12931
```

Lesson 5.2: Building functions out of other functions

Once you have a small, focused function, you can use it as a building block for larger ones. Here's a function that uses our earlier `spread_info` to summarize a column:

```
spread_info <- function(x) {  
  list(  
    smallest = min(x, na.rm = TRUE),  
    largest = max(x, na.rm = TRUE),  
    range_size = max(x, na.rm = TRUE) - min(x, na.rm = TRUE)  
  )  
}
```

```
column_spread <- function(df, col) {  
  spread_info(df[[col]])  
}
```

```
column_spread(mtcars, "mpg")
```

```
## $smallest
```

```
## [1] 10.4
```

```
##
```

```
## $largest
```

```
## [1] 33.9
```

```
##
```

```
## $range_size
```

```
## [1] 23.5
```

```
column_spread(airquality, "Wind")
```

```
## $smallest
```

```
## [1] 1.7
```

```
##
```

```
## $largest
```

```
## [1] 20.7
```

```
##
```

```
## $range_size
```

```
## [1] 19
```

The new function pulls the column out of the data frame and hands it off to the existing function. This is **composition** — building bigger tools from smaller ones. It keeps each function small enough to reason about and reuse.

Exercise 5.2

Write `column_summary(df, col)` that uses your `vec_summary` function from Exercise 3.4 internally — pull the column out of the data frame, then pass it to `vec_summary`. Test it on `mtcars` with `"hp"`.

```
vec_summary <- function(x) {
  list(
    mean = mean(x, na.rm = TRUE),
    median = median(x, na.rm = TRUE),
    min = min(x, na.rm = TRUE),
    max = max(x, na.rm = TRUE),
    n_missing = sum(is.na(x))
  )
}

column_summary <- function(df, col) {
  vec_summary(df[[col]])
}

column_summary(mtcars, "hp")
```

```
## $mean
## [1] 146.6875
##
## $median
## [1] 123
##
## $min
## [1] 52
##
## $max
## [1] 335
##
## $n_missing
## [1] 0
```

Lesson 5.3: Filtering rows based on a condition

You can write functions that subset a data frame based on values in a column:

```
top_quartile <- function(df, col) {
  cutoff <- quantile(df[[col]], 0.75, na.rm = TRUE)
  df[df[[col]] >= cutoff, ]
}

# Cars in the top quartile of horsepower
top_quartile(mtcars, "hp")
```

```
##           mpg cyl  disp  hp drat    wt  qsec vs am gear carb
## Duster 360    14.3   8 360.0 245 3.21 3.570 15.84 0 0   3    4
## Merc 450SE    16.4   8 275.8 180 3.07 4.070 17.40 0 0   3    3
## Merc 450SL    17.3   8 275.8 180 3.07 3.730 17.60 0 0   3    3
## Merc 450SLC   15.2   8 275.8 180 3.07 3.780 18.00 0 0   3    3
## Cadillac Fleetwood 10.4   8 472.0 205 2.93 5.250 17.98 0 0   3    4
```

```
## Lincoln Continental 10.4 8 460.0 215 3.00 5.424 17.82 0 0 3 4
## Chrysler Imperial 14.7 8 440.0 230 3.23 5.345 17.42 0 0 3 4
## Camaro Z28 13.3 8 350.0 245 3.73 3.840 15.41 0 0 3 4
## Ford Pantera L 15.8 8 351.0 264 4.22 3.170 14.50 0 1 5 4
## Maserati Bora 15.0 8 301.0 335 3.54 3.570 14.60 0 1 5 8
```

Note the subsetting syntax: `df[df[[col]] >= cutoff, ]` keeps rows where the condition is true (the `,` with nothing after it means “all columns”).

Exercise 5.3

Write a function called `above_average(df, col)` that returns the subset of rows where the value in column `col` is above the mean of that column (NAs removed). Test it on `mtcars` with “mpg” — you should get back the more fuel-efficient cars.

```
above_average <- function(df, col) {
  average <- mean(df[[col]], na.rm = TRUE)
  df[df[[col]] > average, ]
}
```

```
above_average(mtcars, "mpg")
```

```
##      mpg cyl  disp  hp drat   wt  qsec vs am gear carb
## Mazda RX4      21.0   6  160.0 110 3.90 2.620 16.46 0 1   4   4
## Mazda RX4 Wag  21.0   6  160.0 110 3.90 2.875 17.02 0 1   4   4
## Datsun 710     22.8   4  108.0  93 3.85 2.320 18.61 1 1   4   1
## Hornet 4 Drive 21.4   6  258.0 110 3.08 3.215 19.44 1 0   3   1
## Merc 240D      24.4   4  146.7  62 3.69 3.190 20.00 1 0   4   2
## Merc 230       22.8   4  140.8  95 3.92 3.150 22.90 1 0   4   2
## Fiat 128       32.4   4   78.7  66 4.08 2.200 19.47 1 1   4   1
## Honda Civic    30.4   4   75.7  52 4.93 1.615 18.52 1 1   4   2
## Toyota Corolla 33.9   4   71.1  65 4.22 1.835 19.90 1 1   4   1
## Toyota Corona  21.5   4  120.1  97 3.70 2.465 20.01 1 0   3   1
## Fiat X1-9      27.3   4   79.0  66 4.08 1.935 18.90 1 1   4   1
## Porsche 914-2  26.0   4  120.3  91 4.43 2.140 16.70 0 1   5   2
## Lotus Europa   30.4   4   95.1 113 3.77 1.513 16.90 1 1   5   2
## Volvo 142E     21.4   4  121.0 109 4.11 2.780 18.60 1 1   4   2
```

Lesson 5.4: NULL defaults for “compute later”

Sometimes a default value depends on the input data and can’t be computed when the function is defined (because the data doesn’t exist yet). The trick is to use `NULL` as a placeholder default and check for it inside the function with `is.null()`:

```
filter_above <- function(df, col, threshold = NULL) {
  if (is.null(threshold)) {
    threshold <- mean(df[[col]], na.rm = TRUE) # default to the column's mean
  }
  df[df[[col]] > threshold, ]
}

# Use the default (column mean):
result1 <- filter_above(mtcars, "mpg")
nrow(result1)
```

```
## [1] 14
# Override with a specific threshold:
result2 <- filter_above(mtcars, "mpg", threshold = 25)
nrow(result2)
```

```
## [1] 6
```

The `is.null()` check tells you whether the caller supplied a value or not. This is a common pattern in R — you'll see it in many built-in functions.

Exercise 5.4

Write `above_threshold(df, col, threshold = NULL)` that returns the subset of rows where `col` exceeds `threshold`. If `threshold` is `NULL`, default to the column's mean.

Test it both ways: with an explicit threshold and with the default.

```
above_threshold <- function(df, col, threshold = NULL) {
  if(is.null(threshold)) {
    threshold <- mean(df[[col]], na.rm = TRUE)
  }
  df[df[[col]] > threshold, ]
}

result3 <- filter_above(mtcars, "hp")
nrow(result3)
```

```
## [1] 15
result4 <- filter_above(mtcars, "hp", threshold = 180)
nrow(result4)
```

```
## [1] 7
```

6. Advanced Topics

Lesson 6.1: Lazy evaluation

R only evaluates function arguments **when they're actually used** in the body. This means you can sometimes call a function without supplying all the arguments — as long as the missing one is never referenced:

```
quirky <- function(p, q) {
  p * 10          # only `p` is used; `q` is never touched
}

quirky(7)         # works, even though we didn't supply q
```

```
## [1] 70
```

If we tried this on a function that uses `q`, R would error when it hit the line that references `q`. The key idea: R doesn't check whether arguments exist up front — it only checks when each one is actually needed.

Tip: Lazy evaluation is more of a quirk than a feature to rely on. Don't write functions that depend on it — always supply all required arguments. But it's good to know about, because it explains some confusing error messages where part of the function output appears before the error.

Exercise 6.1

Write a function `lazy_demo(a, b)` that prints `a` but never references `b`. Then call it with just one argument: `lazy_demo(5)`. It should work without error.

```
lazy_demo <- function(a, b) {  
  a  
}  
lazy_demo(5)
```

```
## [1] 5
```

Lesson 6.2: The ... argument

The ... (three dots) argument is a special placeholder that lets your function accept additional arguments and pass them along to another function. This is useful when you're building a wrapper around an existing function and don't want to copy its entire argument list.

```
verbose_max <- function(x, ...) {  
  cat("Finding the max...\n")  
  result <- max(x, ...)           # pass `...` through to max()  
  cat("Done!\n")  
  result  
}
```

```
# Now we can pass na.rm = TRUE through to max(),  
# even though our function doesn't declare na.rm:  
verbose_max(airquality$Ozone, na.rm = TRUE)
```

```
## Finding the max...
```

```
## Done!
```

```
## [1] 168
```

The caller can use any argument that `max()` accepts (`na.rm`, etc.), and ... will forward them on. You'll see ... used in many built-in R functions, often as the very first argument to functions like `paste()` and `cat()` that can accept any number of inputs.

Exercise 6.2

Write `my_mean(x, ...)` that calls `mean(x, ...)` internally. Test it with `my_mean(airquality$Ozone, na.rm = TRUE)`.

```
my_mean <- function(x, ...) {  
  mean(x, ...)  
}  
my_mean(airquality$Ozone, na.rm = TRUE)
```

```
## [1] 42.12931
```

Exercise 6.3

Write a function `describe_column(df, col, ...)` that prints the column name and then calls `summary(df[[col]], ...)`. The ... allows callers to pass additional arguments to `summary()`. Test it on `mtcars` with "mpg".

```
describe_column <- function(df, col, ...) {
  cat(col, "\n")
  summary(df[[col]], ...)
}
```

```
describe_column(mtcars, "mpg")
```

```
## mpg
```

```
##      Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
##    10.40   15.43   19.20   20.09   22.80   33.90
```

Mini Challenges

These tie everything together. Each one combines multiple concepts from above.

Challenge 1: BMI calculator with units

Write `bmi_calc(weight, height, units = "imperial")` that calculates BMI:

- If `units = "imperial"`: weight in pounds, height in inches. Formula: $(\text{weight} / \text{height}^2) * 703$
- If `units = "metric"`: weight in kg, height in meters. Formula: $\text{weight} / \text{height}^2$

The function should return a **named list** with two elements: `bmi` (the numeric value) and `category` (one of "underweight", "normal", "overweight", "obese" based on standard cutoffs: <18.5, 18.5–25, 25–30, 30).

If `units` is anything other than "imperial" or "metric", halt with `stop("Invalid units")`.

Test it with both unit systems.

Tip: `stop("message")` halts execution and reports an error. It's how you signal that something is wrong with the inputs, instead of returning a wrong answer silently.

```
bmi_calc <- function(weight, height, units = "imperial") {
  if (units == "imperial") {
    bmi <- (weight/height^2) * 703
  } else if (units == "metric"){
    bmi <- weight/height^2
  } else {
    stop("Invalid units")
  }

  category <- if (bmi < 18.5) {
    "underweight"
  } else if (bmi >= 18.5 & bmi < 25) {
    "normal"
  } else if (bmi >= 25 & bmi < 30) {
    "overweight"
  } else {
    "obese"
  }
  list(
    bmi = bmi,
    category = category
  )
}
```

```

}

bmi_calc(150, 67)                # imperial, should be ~23.5, normal

## $bmi
## [1] 23.49076
##
## $category
## [1] "normal"

bmi_calc(68, 1.73, units = "metric")  # metric, should be ~22.7, normal

## $bmi
## [1] 22.72044
##
## $category
## [1] "normal"

bmi_calc(100, 70)                # imperial, underweight

## $bmi
## [1] 14.34694
##
## $category
## [1] "underweight"

bmi_calc(300, 65)                # imperial, obese

## $bmi
## [1] 49.91716
##
## $category
## [1] "obese"

bmi_calc(150, 67, units = "banana")  # should trigger your stop() error

## Error in `bmi_calc()`:
## ! Invalid units

```

Challenge 2: Apply to multiple columns

Using your `vec_summary` function from Exercise 3.4, write a `for` loop that applies it to **every column** of `mtcars` and stores the results in a named list called `all_summaries`. Then access `all_summaries$mpg$mean` to verify it worked.

Hint: `colnames(mtcars)` gives you all the column names to loop over.

```

vec_summary <- function(x) {
  list(
    mean = mean(x, na.rm = TRUE),
    median = median(x, na.rm = TRUE),
    min = min(x, na.rm = TRUE),
    max = max(x, na.rm = TRUE),
    n_missing = sum(is.na(x))
  )
}

```

```

all_summaries <- list()
for (col in colnames(mtcars)) {
  all_summaries[[col]] <- vec_summary(mtcars[[col]])
}
all_summaries$mpg$mean

## [1] 20.09062

```

Challenge 3: Smart summary

Write a function called `smart_summary(x)` that adapts its behavior to the input type:

- If `x` is numeric, return `vec_summary(x)` (your function from Exercise 3.4)
- If `x` is character or factor, return a `table()` of the values
- If `x` is logical, return the count of `TRUE` and `FALSE` values

Use `if/else if` and the `is.numeric()`, `is.character()`, `is.factor()`, `is.logical()` test functions.

Test it on `iris$Sepal.Length` (numeric), `iris$Species` (factor), and a logical vector you make yourself.

```

vec_summary <- function(x) {
  list(
    mean = mean(x, na.rm = TRUE),
    median = median(x, na.rm = TRUE),
    min = min(x, na.rm = TRUE),
    max = max(x, na.rm = TRUE),
    n_missing = sum(is.na(x))
  )
}

smart_summary <- function(x) {
  if (is.numeric(x)) {
    vec_summary(x)
  } else if (is.character(x) | is.factor(x)) {
    table(x)
  } else if (is.logical(x)) {
    table(x)
  }
}

smart_summary(iris$Sepal.Length)

## $mean
## [1] 5.843333
##
## $median
## [1] 5.8
##
## $min
## [1] 4.3
##
## $max
## [1] 7.9

```

```
##
## $n_missing
## [1] 0

smart_summary(iris$Species)

## x
##      setosa versicolor  virginica
##      50      50      50

smart_summary(c(TRUE, FALSE, TRUE, TRUE, FALSE))

## x
## FALSE  TRUE
##      2      3
```

Challenge 4: Refactor

Here's some working code that's repetitive — the same calculation done twice on different columns:

```
ozone_above_50 <- airquality$Ozone[!is.na(airquality$Ozone) & airquality$Ozone > 50]
ozone_above_50_count <- length(ozone_above_50)
ozone_above_50_mean <- mean(ozone_above_50)

wind_above_15 <- airquality$Wind[!is.na(airquality$Wind) & airquality$Wind > 15]
wind_above_15_count <- length(wind_above_15)
wind_above_15_mean <- mean(wind_above_15)
```

Refactor it into a single function `above_threshold_stats(x, threshold)` that takes a vector and threshold and returns a named list with `count` and `mean` of values above the threshold. Then reproduce the same results in just two function calls.

Tip: Refactoring repetitive code into functions is one of the highest-value skills in programming. Any time you find yourself copying code with small changes, that's a sign a function should exist.

```
above_threshold_stats <- function(x, threshold) {
  above <- x[!is.na(x) & x > threshold]
  list(
    mean = mean(above, na.rm = TRUE),
    count = sum(!is.na(above))
  )
}
```

```
above_threshold_stats(airquality$Ozone, 50)
```

```
## $mean
## [1] 85.88235
##
## $count
## [1] 34
```

```
above_threshold_stats(airquality$Wind, 15)
```

```
## $mean
## [1] 17.16
##
## $count
```



```
## [1] 10
```

Challenge 5: A complete pipeline

Combine everything. Write `analyze_column(df, col, threshold = NULL)` that:

1. Pulls the column from the data frame.
2. Counts how many NAs are in it.
3. If `threshold` is `NULL`, sets it to the column's mean (NAs removed).
4. Counts how many non-missing values are above the threshold.
5. Returns a named list with: `column_name`, `n_missing`, `threshold_used`, `n_above_threshold`, and `pct_above_threshold` (as a percentage of non-missing values).

Test it on `airquality` with "Ozone" (let it use the default threshold) and on `mtcars` with "hp" and `threshold = 200`.

```
analyze_column <- function(df, col, threshold = NULL) {  
  total <- df[[col]]  
  missing <- sum(is.na(df[[col]]))  
  if (is.null(threshold)){  
    threshold <- mean(df[[col]], na.rm = TRUE)  
  }  
  above <- !is.na(total) & total > threshold  
  count <- sum(above)  
  list(  
    column_name = col,  
    n_missing = missing,  
    threshold_used = threshold,  
    n_above_threshold = count,  
    pct_above_threshold = count / (length(total) - missing)  
  )  
}
```

```
analyze_column(airquality, "Ozone")
```

```
## $column_name  
## [1] "Ozone"  
##  
## $n_missing  
## [1] 37  
##  
## $threshold_used  
## [1] 42.12931  
##  
## $n_above_threshold  
## [1] 44  
##  
## $pct_above_threshold  
## [1] 0.3793103
```

```
analyze_column(mtcars, "hp", threshold = 200)
```

```
## $column_name  
## [1] "hp"  
##
```

```
## $n_missing
## [1] 0
##
## $threshold_used
## [1] 200
##
## $n_above_threshold
## [1] 7
##
## $pct_above_threshold
## [1] 0.21875
```